МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

# ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине

«Сервис-ориентированная архитектура»

Вариант: 89

Работу выполнили:
Студенты группы P3411
**Болорболд Аригуун**
**Орчиков Даниил Валерьевич**

Лектор:
**Цопа Евгений Алексеевич**
Практик:
**Кривоносов Егор Дмитриевич**

г. Санкт-Петербург
2025

# Содержание

1. Текст задания . . . . . 3
2. Выполнение . . . . . 4
   2.1. Исходный код . . . . . 4
   2.2. Модификация условии . . . . . 4
   2.3. Клиентское приложение . . . . . 4
   2.4. Проброс портов: . . . . . 4
   2.5. Создание сертификатов и настройка SSL . . . . . 4
   2.6. Особенности при работе: . . . . . 5
3. Вывод . . . . . 6

# 1. Текст задания

На основе разработанной в рамках лабораторной работы 1 спецификации реализовать два веб-сервиса и использующее их API клиентское приложение.

**Требования к реализации и развёртыванию сервисов:**

- Первый («вызываемый») веб-сервис должен быть реализован на фреймворке `JAX-RS` и развёрнут в окружении под управлением сервера приложений `Payara`.
- Второй веб-сервис должен быть реализован на фреймворке `Spring MVC REST`, развёрнут в окружении под управлением сервера приложений `WildFly` и вызывать REST API первого сервиса.
- **Для обоих сервисов необходимо реализовать все функции, задокументированные в API, в строгом соответствии со спецификацией!**
- Доступ к обоим сервисам должен быть реализован с по протоколу https с самоподписанным сертификатом сервера. Доступ к сервисам посредством http без шифрования должен быть запрещён.

**Требования к клиентскому приложению:**

- Клиентское приложение может быть написано на любом веб-фреймворке, который можно запустить на сервере helios.
- Приложение должно обеспечить полный набор возможностей, предоставляемых API обоих сервисов — включая сортировку, фильтрацию и постраничный вывод элементов коллекции.
- Приложение должно преобразовывать передаваемые сервисами данные в человеко-читаемый вид — параграф текста, таблицу и т.д.
- Клиентское приложение должно информировать пользователя об ошибках, возникающих на стороне сервисов, в частности, о том, что сервису были отправлены невалиданые данные.

Оба веб-сервиса и клиентское приложение должны быть развёрнуты на сервере helios.

# 2. Выполнение

## 2.1. Исходный код

GitHub: https://github.com/17starplatinum/SOAJerseyClub

## 2.2. Модификация условии

В связи с тем, что исходный вариант из первой лабораторной работы сам по себе не имел логического объяснения для одной из функции, нами было принято решение добавить ещё одну сущность, чтобы подправить условие. Это привело к тому, что оба сервиса теперь могут вызвать другого, и оба сервиса имеет собственную базу данных. Так как это невозможно выполнить в условиях факультетского сервера helios, под «собственными БД» здесь разумеется работа через разных схем (s37XXXX и s36YYYY). Фактически они являются частью одного и того же БД, но так как их данные находятся автономно от друг друга, реализация этого условия через схем является достойным вариантом симуляции реального СОА.

## 2.3. Клиентское приложение

Стек: React + Vite + TS
https://se.ifmo.ru/~s367452/soa-front

## 2.4. Проброс портов:

```
ssh -p 2222 s367452@se.ifmo.ru -L 15478:helios:15478 -L
15479:helios:15479
```

## 2.5. Создание сертификатов и настройка SSL

### 2.5.1. Для WildFly

1. ```
   keytool -genkeypair -alias wildfly -keyalg RSA -keysize 2048 \
     -dname "CN=localhost, OU=ITMO, O=SOA, L=SPB, ST=RU, C=RU" \
     -validity 3650 \
     -keystore wildfly.keystore \
     -storepass secret \
     -keypass secret
   ```
2. Кладем `wildfly.keystore` в `$WILDFLY_HOME/standalone/configuration/`
3. ```
   keytool -exportcert -alias wildfly -keystore wildfly.keystore -
   file wildfly-cert.crt
   ```
4. ```
   keytool -importcert -file payara.crt -keystore truststore.jks -
   alias wildfly -storepass changeit
   ```
5. ```
   ./standalone.sh \
   -Djavax.net.ssl.trustStore="$WILDFLY_HOME/standalone/
   configuration/truststore.jks" \
   -Djavax.net.ssl.trustStorePassword=changeit
   ```

### 2.5.2. Для Payara Micro

1. ```
   keytool -genkeypair -alias payara -keyalg RSA -keysize 2048 \
     -dname "CN=localhost, OU=ITMO, O=SOA, L=SPB, ST=RU, C=RU" \
     -validity 3650 \
     -keystore payara.keystore \
     -storepass secret \
     -keypass secret
   ```
2. ```
   keytool -exportcert -alias payara -keystore payara.keystore -
   file payara.crt
   ```
3. ```
   keytool -importcert -file wildfly-cert.crt -keystore
   truststore.jks -alias payara -storepass changeit
   ```
4. ```
   java -Djavax.net.ssl.keyStore=payara.keystore \
   -Djavax.net.ssl.trustStore=truststore.jks \
   -Djavax.net.ssl.keyStorePassword=secret \
   -Djavax.net.ssl.trustStorePassword=changeit \
   -Duser.language=en -Duser.region=US
    -jar payara-micro-6.2025.10.jar \
   --sslPort 15478 --sslCert payara --deploy ./HumanBeingModule.war
   --contextroot /
   ```

## 2.6. Особенности при работе:

1. Payara Micro принимает локаль той машины, на которую он работает. То есть, если его запустить на `helios` (где русский язык по умолчанию), то оно будет обратно отправить метаданные (заголовки) на этом же локале. Но так как оно работает на базе Eclipse Grizzly (HTTP/2), то ему требуется использовать алгоритм Хаффмана для сжатия информации, а сам алгоритм не умеет работать с кириллицей. Внизу прикреплен пример этого поведения.

```
[2025-11-17T15:57:26.054+0300] [] [WARNING] []
[org.glassfish.grizzly.filterchain.DefaultFilterChain] [tid:
_ThreadID=135 _ThreadName=http-thread-pool::https-listener(1)]
[timeMillis: 1763384246054] [levelValue: 900] [[
  GRIZZLY0013: Exception during FilterChain execution
java.lang.IllegalArgumentException: char=1087 # буква 'П' от даты
        at
org.glassfish.grizzly.http2.hpack.Huffman.codeOf(Huffman.java:532)
        at
        ...
```

Пришлось вручную изменить локал на английскую, чтобы решить эту проблему.

2. Для более эффективной отладки можно на локальном устройстве запустить ошибочный сервис, а другого на гелиосе. Таким образом мы

более эффективным и практичным образом застали принципы работа СОА. Также базы данных были совсем иные и не связанные с друг другом, что также закрепил для нас этот факт.

3. Для убеждения в корректности и согласованности данных мы написали модульные тесты для сущностей, DTO и сервисов.

## 3. Вывод

В рамках этой работы мы реализовали сервис-ориентированную архитектуру на практике. По мере работы мы распознавали принципы СОА и их проявление в работе. Также мы ознакомились с обеспечением шифрованного подключения к API, реализовав его через самоподписанные сертификаты. Ещё в связи с особым условием нам также приходилось настраивать подключение сервисов в одну базу данных через разные схемы. Это на практике является антипаттерном, но в кафедровом сервере нет такой возможности, поэтому в учебных целях такая практика подходит для нас.